Before the elbow: WCSS drops quickly -> clusters become much better. After the elbow: WCSS drops slowly -> extra clusters add little value and may lead to overfitting.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline
```

```
# import kagglehub
# path = kagglehub.dataset_download("pathaksharma/mall-customer")
data=pd.read_csv('/content/sample_data/Mall_Customers.csv')
data.head()
```

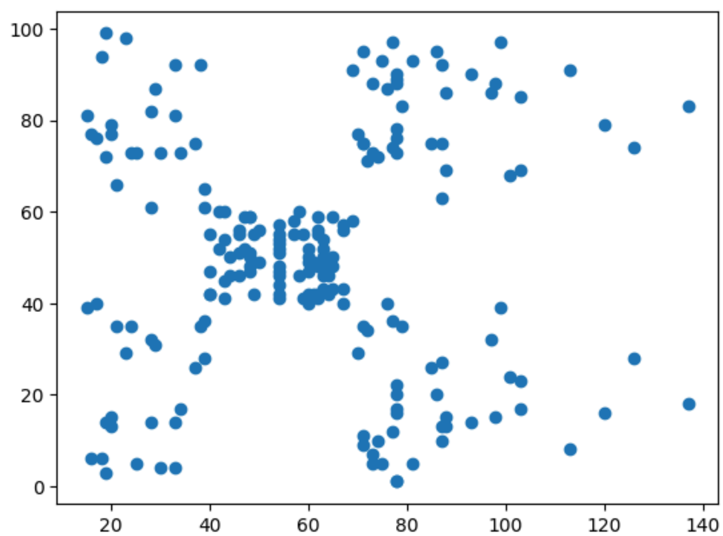
	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

```
data.shape
```

```
(200, 5)
```

```
x=data.iloc[:, 3].values
y=data.iloc[:, 4].values
plt.scatter(x, y)
```

```
<matplotlib.collections.PathCollection at 0x79ab66b51a30>
```




```
x=data.iloc[:, 3:].values
x
```

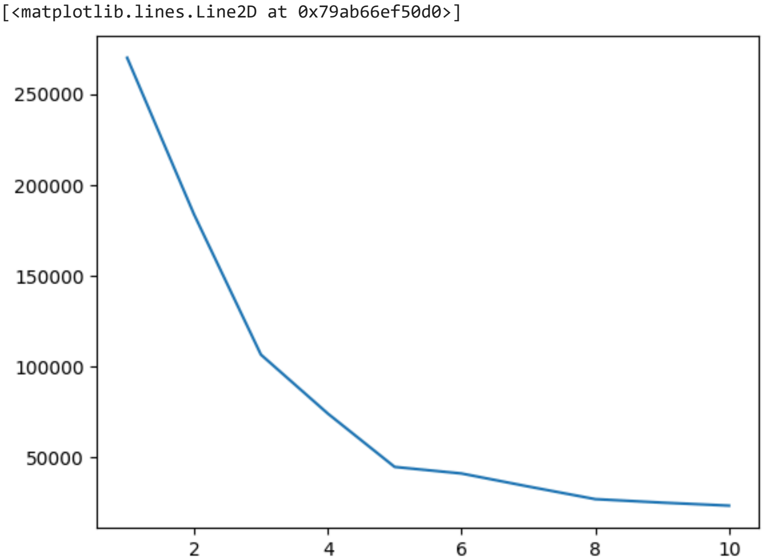
```
array([[ 15,  39],
       [ 15,  81],
       [ 16,   6],
       [ 16,  77],
       [ 17,  40],
       [ 17,  76],
       [ 18,   6],
       [ 18,  94],
       [ 19,   3],
       [ 19,  72],
       [ 19,  14],
       [ 19,  99],
       [ 20,  15],
       [ 20,  77],
```

```
       [ 20,  13],
       [ 20,  79],
       [ 21,  35],
       [ 21,  66],
       [ 23,  29],
       [ 23,  98],
       [ 24,  35],
       [ 24,  73],
       [ 25,   5],
       [ 25,  73],
       [ 28,  14],
       [ 28,  82],
       [ 28,  32],
       [ 28,  61],
       [ 29,  31],
       [ 29,  87],
       [ 30,   4],
       [ 30,  73],
       [ 33,   4],
       [ 33,  92],
       [ 33,  14],
       [ 33,  81],
       [ 34,  17],
       [ 34,  73],
       [ 37,  26],
       [ 37,  75],
       [ 38,  35],
       [ 38,  92],
       [ 39,  36],
       [ 39,  61],
       [ 39,  28],
       [ 39,  65],
       [ 40,  55],
       [ 40,  47],
       [ 40,  42],
       [ 40,  42],
       [ 42,  52],
       [ 42,  60],
       [ 43,  54],
       [ 43,  60],
       [ 43,  45],
       [ 43,  41],
       [ 44,  50],
       [ 44,  46],
```

```
from sklearn.cluster import KMeans
wcss=[]
for i in range(1, 11):
  km=KMeans(n_clusters=i, init='k-means++', random_state=42)
  km.fit(x)
  wcss.append(km.inertia_)
```

```
plt.plot(range(1,11), wcss)
```

```
[<matplotlib.lines.Line2D at 0x79ab66ef50d0>]
```



```
km=KMeans(n_clusters=5)
```

```
y_means=km.fit_predict(x)
y_means
```

```
array([4, 3, 4, 3, 4, 3, 4, 3, 4, 3, 4, 3, 4, 3, 4, 3, 4, 3, 4, 3, 4, 3,
       4, 3, 4, 3, 4, 3, 4, 3, 4, 3, 4, 3, 4, 3, 4, 3, 4, 3, 4, 3, 4, 1,
       4, 3, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 2, 0, 1, 0, 2, 0, 2, 0,
       1, 0, 2, 0, 2, 0, 2, 0, 2, 0, 1, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0,
       2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0,
       2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0,
       2, 0], dtype=int32)
```

```
x[y_means==0,0]
```

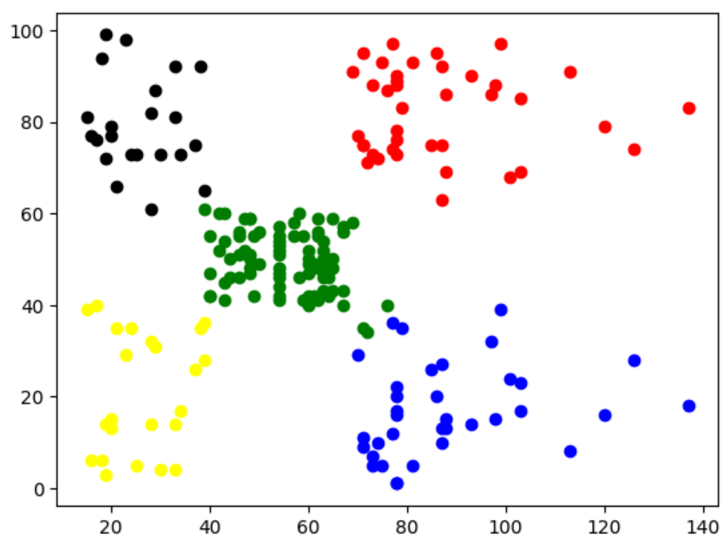
```
array([ 69,  70,  71,  71,  71,  72,  73,  73,  74,  75,  76,  77,  77,
        78,  78,  78,  78,  78,  78,  79,  81,  85,  86,  87,  87,  87,
        88,  88,  93,  97,  98,  99, 101, 103, 103, 113, 120, 126, 137])
```

```
x[y_means==0,1]
```

```
array([91, 77, 95, 75, 75, 71, 88, 73, 72, 93, 87, 97, 74, 90, 88, 76, 89,
       78, 73, 83, 93, 75, 95, 63, 75, 92, 86, 69, 90, 86, 88, 97, 68, 85,
       69, 91, 79, 74, 83])
```

```
plt.scatter(x[y_means==0,0], x[y_means==0,1], color="red")
plt.scatter(x[y_means==1,0], x[y_means==1,1], color="green")
plt.scatter(x[y_means==2,0], x[y_means==2,1], color="blue")
plt.scatter(x[y_means==3,0], x[y_means==3,1], color="black")
plt.scatter(x[y_means==4,0], x[y_means==4,1], color="yellow")
```

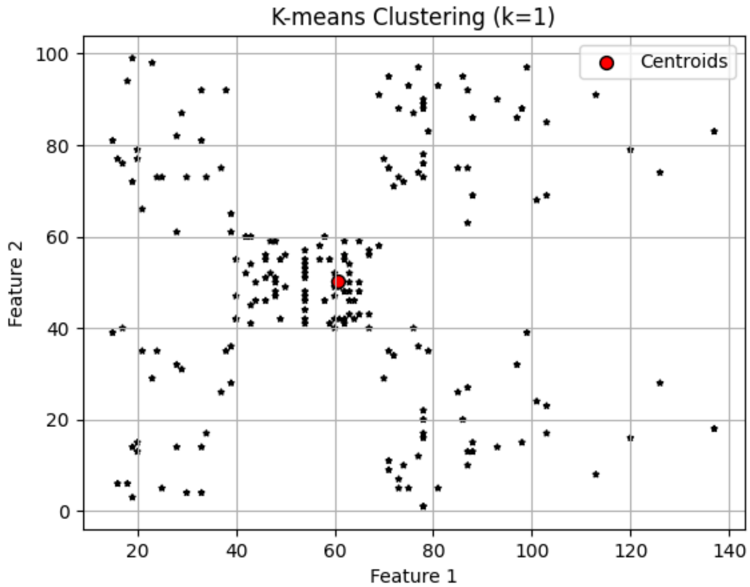
<matplotlib.collections.PathCollection at 0x79ab66cb4620>

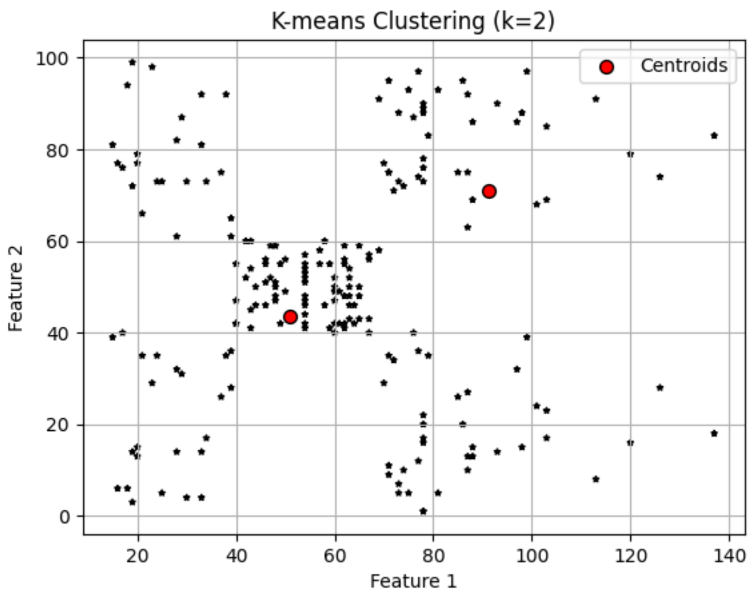


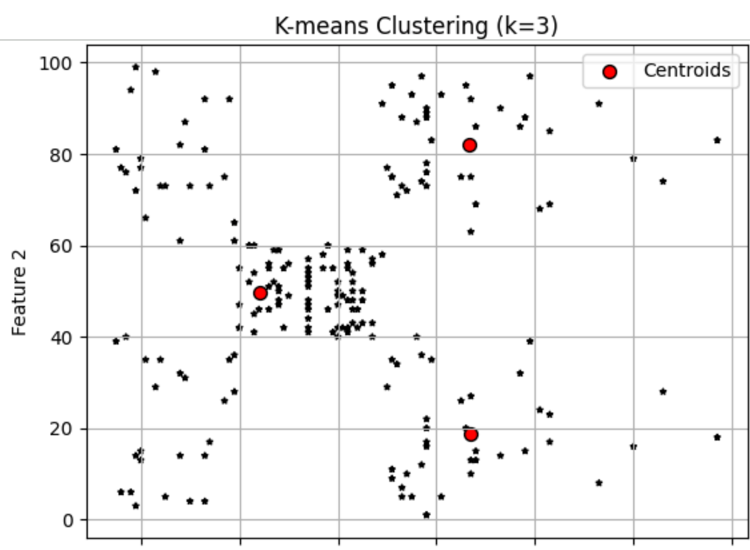

```
k_range = range(1, 6)

for k in k_range:
    km = KMeans(n_clusters=k, init='k-means++', random_state=42)
    y_km = km.fit_predict(x)

    plt.scatter(x[:, 0], x[:, 1], c=y_km, cmap='viridis', marker='*', edgecolor='k', s=10)
    plt.scatter(km.cluster_centers_[:, 0], km.cluster_centers_[:, 1],
                s=50, c='red', label='Centroids', edgecolor='k')
    plt.title(f'K-means Clustering (k={k})')
    plt.xlabel('Feature 1')
    plt.ylabel('Feature 2')
    plt.legend()
    plt.grid()
    plt.show()
```


K-means Clustering (k=1)
Centroids
Feature 1
Feature 2


K-means Clustering (k=2)
Centroids
Feature 1
Feature 2


K-means Clustering (k=3)
Centroids
Feature 2